

Mini Project: Diagnose and Fix a Broken

RAG Retrieval

Time: ~60 minutes

What you'll do

You'll take a small PDF, run it through a RAG pipeline you've already built, and find a case where retrieval looks like it works but actually doesn't. Then you'll diagnose why, and try a minimal fix.

What you need

1. A short PDF (5-15 pages) that contains at least one table or structured list mixed with regular prose — for example, a resume with a skills table, a product spec sheet, a small report with a data table, or a syllabus with a grading breakdown table.
2. Your existing chunking script (fixed-size or semantic — whichever you already have working).
3. No new packages, no RAGAS.

Part 1 — Questions (answer in a few sentences each, no code needed)

Q1. Explain, in your own words, why the vector database (Chroma) has no influence over how a document gets chunked. What is Chroma's actual job in the pipeline?

→ Chroma has no influence over how a document is chunked because chunking is performed before the document is added to the vector database. First, the source document is divided into smaller chunks using a text splitter. These chunks are then converted into embeddings using an embedding model. Chroma's job is to store the chunks and their embeddings and perform similarity search to retrieve the most relevant chunks when a user asks a question. Therefore, Chroma is responsible for **storage and retrieval**, while the **text splitter controls chunking**.

Q2. Your document has a table. Explain why running it through RecursiveCharacterTextSplitter with a fixed chunk_size is risky for that table specifically, even if the table is small.

→ text splitter splits the document according to fixed chunk size. creates one chunk of size x. as soon as the chunk is created of required size it breaks down. it does not see the context, meaning, text structure of the chunk. that's why the content gets mixed with the neighbouring chunk and while we are trying to get response for the specific query we are unable to get it. sometimes we get the expected page or chunk but unable to retrieve the expected context and if we talk about the table specifically, table may be broken down more than one chunk and retriever gets failed to retrieve the information from table related to query.

Q3. Explain the difference between BM25 (keyword) search and vector (semantic) search using one example question from YOUR chosen document where you think each one would individually struggle.

→ BM25 is an algorithm used to retrieve the response for a query based on exact matching keywords and vector search is used to find chunks for a user query based on semantic similarity. I tested a query,

For ex. test query = "What is the title of Experiment no.9 in Practical Paper-I?"

The source document contains two practical papers I & II. both of them contain experiment no.9, so the **BM25 retriever needs the exact matching keywords** to retrieve the correct information otherwise it will struggle to find expected chunk. **Vector search is based on semantic similarity** so it

can search even if the wording in a query is different .however it will also struggle to find experiment no.9 as both the practical papers contains same experiment information.

Q4. True or false, with justification: "If a retrieval system returns the correct page number, it retrieved the correct information." Think about what else might be on that page besides the answer.

--→False.I test the query which is related to the table.Retriever gives the same expected page as chunk of the correct answer but actually its not giving the expected response for a query.However last few lines of the page which are not part of the table are retrieved in the chunk.so the table has been split into chunks,and the embedding search is not recognizing the exact relationship between them.

Q5. You ask your RAG system: "What are the top 3 products by revenue in Q3?" and the table with that exact data exists in your document, but the retriever returns a paragraph that mentions "Q3 revenue grew steadily" instead. Walk through what likely happened to the table during chunking, and why the paragraph might have scored better in similarity search than the table itself.

→fixed-size chunking splits the document according to the chunk size not according to the context.so the tables may be split across chunks.Due to which we may lose content and context of that table.so that retriever may gets failed to retrieve "the top 3 products by revenue in Q3".As the vector search searches the embeddings based on semantic similarity,it may give the paragraph that mentions "Q3 revenue grew steadily".

Paragraph might have scored better in similarity search than the itself because the paragraph contains the words which are in a query so there is semantic similarity between the query and the paragraph.the table may be split into two or more chunks so it is difficult to get expected response from one chunk.

Q6. A teammate says: "I increased k from 3 to 10 and now the right chunk shows up somewhere in the results, so the problem is fixed." Do you agree this counts as "fixed"?

What's the trade-off of just raising k instead of fixing the underlying chunking?

→No,increasing k from 3 to 10 will not fix the problem of getting right chunk.It will only increase the chances of getting right chunk somewhere in the result but it will also give more irrelevant chunks that will increase the amount of information that is processed by LLM.so the right solution is use the chunking strategy that will retrieve the right chunk ranked highly in first place instead of increasing k.

Q7. You're chunking a document with both a glossary table (term → definition) and a FAQ section (question → answer paragraphs). Would you use the same chunking strategy for both sections, or different ones for each? Justify your answer.

→The glossary table and FAQ section may be structured differently.table may have columns,rows and related content.FAQ section may have questions ,answers,paragraphs,etc.so I can't use same chunking strategy for both sections.i have to use different chunking strategy so that I am able to retrieve the correct information whenever I will test query related to both sections.

Q8. A user asks a question using casual, everyday wording, but your document uses formal technical terminology to describe the same concept. Name one retrieval strategy from what we've covered that directly targets this problem, and briefly explain how it works.

→ Vector search/semantic retrieval strategy search the embeddings based on similarity search. it doesn't require exact wording in the query as in the document.

For ex. test_query=" "What are the distributions for which probabilities are computed using R software in Practical Paper-I?"

Here, exact keywords or distributions name are not mentioned in the query. Still semantic retrieval Recognise the relationship even though wording is different.

Q9. Your retrieval system returns the exact correct chunk in first place (rank 1) for a test question. Your teammate says this means the chunking strategy is definitely good and you don't need to test anything else. What would you say to push back on this conclusion?

→ Getting exact chunk in first place for a particular question doesn't mean that the chunking strategy is good. To check retrieval performance, we have to test multiple types of questions based on document structure. I have tested 10 questions to evaluate structure aware chunking is working fine or not. I got Hit Rate@5 of 90% and an MRR of 0.658, but it still missed some queries. This shows that why multiple test cases are necessary.

Part 2 — Practical

➤ **Step 1** — Pick a target (5 min). Find one piece of information in your PDF that lives inside a table, list, or other structured (non-paragraph) element. Write down:

- The exact question you'd ask to retrieve it

→ test_query = "What is the Title of experiment 9 in Practical Paper-I?"

- The exact correct answer, word for word, from the document

→ Finding summary statistics using summary() and fivenum() functions. Calculate arithmetic mean (A.M.), geometric mean (G.M.), harmonic mean (H.M.), median, mode, quantiles, range, quartile deviation (Q.D.), variance, coefficient of variation (C.V.) (ungrouped data) using R software.

- The page number it's on

→ Page no.10

➤ **Step 2** — Run baseline retrieval (10 min). Using your chunking script:

1. Chunk the document
2. Store it in Chroma
3. Run your target question through the retriever with k=3
4. Print the retrieved chunks

Record: did the correct page show up? Did the correct chunk actually contain your target answer, or just something nearby it?

Step 3 — Diagnose (5 min). If retrieval got it wrong (or got the right page but wrong content), check:

Was the table/list cut apart by the chunker? Look at the actual printed chunk text.

Even if intact, does the chunk read as a coherent sentence, or is it a bare list of items with no context?

➤ **Step 4** — Try a fix (5 min). Try one small fix and re-test:

Manually rewrite that one chunk as a labeled sentence (e.g. "The top 3 products by Q3 revenue are: X, Y, Z.") and re-embed just that change — does retrieval improve?

Or: reduce chunk_size and see if it changes anything.

Part 3 — Write-up (5 min)

In 3-5 sentences: what specifically broke, why, and what a real fix would look like if this were a production system with hundreds of similar documents — not just a one-off manual patch on this single chunk.

→ Fixed-size chunking strategy split the chunks according to character length instead of logical structure. That's why the chunks lose their context and content, content gets mixed with nearby chunk. So when I tried to retrieve the response for the query mentioned above it was not showing the right chunk. Even when I used Hybrid search with Re-ranking retrieved semantically similar but different/incorrect sections such as practical paper-II even when the question was asked specifically about Practical Paper-I. So the final production solution is that use structure aware chunking, document specific which will preserve the actual structure of source document such as units, tables, experiments, references, semester, etc.

This approach can then be applied consistently across hundreds of similar documents instead of manually correcting individual chunks.